

Build Your App on Replit

A Complete Beginner's Guide

Manual coding · No AI needed · Step-by-step

What you will learn:

1. Getting started & creating your account
2. The Replit interface — every panel explained
3. Writing your first code (HTML !' CSS !' JavaScript)
4. Installing packages with npm and pip
5. Building a React app from scratch
6. Environment variables & keeping secrets safe
7. Deploying your app live on the internet
8. Keyboard shortcuts & pro tips
9. Common errors and how to fix them
10. Full project build checklist

Made for developers who code manually from scratch

Chapter 1: Getting Started

Create your free account and launch your first project

1.1 Create Your Replit Account

1 Go to replit.com

Open any browser and navigate to <https://replit.com>

2 Click "Sign up"

Choose Google, GitHub, or sign up with your email address and a password.

3 Confirm your email

Check your inbox and click the confirmation link Replit sends you.

4 Choose the free plan

Click "Start for free" — the free plan is enough to build and deploy real apps.

1.2 Create Your First Project

1 Click the blue "+ Create Repl" button

Found on the left sidebar or the dashboard home screen.

2 Pick a template

Search for the language you want:

- "HTML, CSS, JS" !' basic website
- "Node.js" !' backend / server
- "React" !' modern web app
- "Python" !' scripts, data, backend

3 Name your project

Type a short name in the "Title" box, e.g. "my-first-app". No spaces — use hyphens.

4 Click "Create Repl"

Replit sets up your project in a few seconds and opens the code editor.

📝 A "Repl" is just what Replit calls a project. One Repl = one application or script.

1.3 The Dashboard Overview

- Home — recent projects and a quick-create button
- My Repls — the full list of everything you have built
- Explore — browse public projects from other developers
- Teams — for collaborating with others (skip this for now)
- Account avatar (top right) — settings, billing, profile

Chapter 2: The Editor — Every Panel Explained

Know exactly what you are looking at

2.1 The Three Main Panels

Left Panel — File Explorer

All your project files and folders live here. It works like Windows Explorer or Mac Finder for your code.

- Click a file !' opens it in the editor
- Right-click a file !' rename, delete, move, duplicate
- Click the + icon !' create a new file or a new folder

Middle Panel — Code Editor

This is where you type your code. Keywords are coloured automatically — that is called syntax highlighting.

- Click in the editor and start typing
- Ctrl + S (Cmd + S on Mac) !' saves the file
- Ctrl + Z !' undo last change
- Ctrl + / !' comment or uncomment a line
- Tab !' indent a line; Shift + Tab !' un-indent

Right Panel — Output / Console / Shell

This panel has multiple tabs:

- Output / Preview !' shows your running app (the live website or printed text)
- Console !' shows messages your code prints; red text = error messages
- Shell !' a real Linux terminal where you type commands directly

2.2 The Top Toolbar Buttons

%¶ Run

!• Start your app — same as pressing Ctrl + Enter

Stop %

!• Stop the running app

Fork

!• Make a copy of someone else's public project

Share

!• Get a sharable link or invite a collaborator

Deploy

!• Publish your app live so anyone can access it

2.3 Essential Shell Commands

The Shell tab is a real Linux terminal. These are the only commands you need to know at first:

```
node --version
CHECK YOUR NODE.JS VERSION
ls
LIST FILES IN THE CURRENT FOLDER
cd src
GO INTO A FOLDER CALLED "SRC"
cd ..
GO BACK UP ONE LEVEL
pwd
SHOW THE CURRENT FOLDER PATH
touch myfile.txt
CREATE A NEW EMPTY FILE
mkdir myFolder
CREATE A NEW FOLDER
```

Chapter 3: Writing Code Manually

Your first HTML page, then CSS and JavaScript

3.1 A Complete HTML Page

If you're using an IDE like VS Code, the templates loaded in the editor will create everything and type it for you. If you're not, you'll have to create everything manually — do not paste it, typing it builds the skill:

3.2 Adding CSS

3.3 Adding JavaScript

Create a new file called "script.js". Type:

Chapter 4: Installing Packages

Add ready-made libraries to your project

4.1 What Is a Package?

A package is code someone else already wrote. Instead of building everything from scratch, you install a package and use it. For example, "express" makes building a web server easy; "axios" makes calling APIs easy.

4.2 npm — Node.js Package Manager

```
$ npm install express
INSTALL A PACKAGE
$ npm install --save-dev nodemon
INSTALL A DEV ONLY PACKAGE
$ npm install axios dotenv lodash
INSTALL MULTIPLE AT ONCE
$ npm uninstall express
REMOVE A PACKAGE
$ npm list --depth=0
SEE ALL INSTALLED PACKAGES
```

Ø=Ü; After npm install, a folder called node_modules appears. Never edit files inside it — it is auto-generated.

4.3 pip — Python Package Manager

```
$ pip install flask
INSTALL A PACKAGE
$ pip install -r requirements.txt
INSTALL FROM A REQUIREMENTS FILE
$ pip list
SEE INSTALLED PACKAGES
```

4.4 The package.json File (Node.js)

This is your project's config file. It is created automatically and lists all packages and scripts.

- "name" — the name of your project
- "scripts" — commands you run with npm run <name>
- "dependencies" — packages needed when the app runs
- "devDependencies" — packages only needed during development

```
$ npm run start
RUN THE "START" SCRIPT
$ npm run dev
RUN THE "DEV" SCRIPT (COMMON FOR LOCAL DEVELOPMENT)
$ npm run build
RUN THE "BUILD" SCRIPT (COMPILES/BUNDLES THE APP)
```

& p Never delete package.json or package-lock.json. They track exactly what your project needs.

Chapter 5: Building a React App

The most popular way to build modern web interfaces

5.1 What Is React?

React is a JavaScript library for building user interfaces. Instead of one big HTML file, you build small reusable pieces called "components". Each component is a JavaScript function that returns HTML-like code called JSX.

5.2 Start a React Project on Replit

1 Click "+ Create Repl"

Go to your dashboard and click the create button.

2 Search for "React"

Select the "React" or "React + Vite" template.

3 Name your project

Example: "my-react-app". Then click Create.

```
import { useState } from 'react';
```

4 Click "Run"

```
function App() {  
  const [count, setCount] = useState(0);
```

Replit installs dependencies automatically and opens a default React page in the Preview.

```
  return (  
    <div className="container" style={{ padding: "50px" }}>
```

5.3 Folder Structure of a React App

- src/ ! all your source code goes here
- src/main.jsx ! the entry point — the starting file React reads first
- src/App.jsx ! the main component — start building here
- public/ ! static files like favicon and the base index.html
- package.json ! project config and dependencies
- vite.config.js ! build tool config (leave it as-is for now)

```
export default App;
```

5.4 Your First React Component

Save (Ctrl+S) the Preview auto-refreshes. Click the button — the count goes up.

5.5 Key React Rules

- Component names MUST start with a capital letter: `function MyButton() {}`
- Return exactly ONE parent element — wrap multiple elements in `<div>` or `<>`
- Use curly braces to insert JS values in JSX: `<p>{myVariable}</p>`
- `useState()` creates a variable React watches — when it changes, the screen updates
- Pass data into components with props: `<Button color="blue" label="Click" />`

Chapter 6: Environment Variables & Secrets

Keep API keys and passwords safe

6.1 Why You Need This

An environment variable is a setting stored outside your code. Use them for sensitive values — API keys, database passwords, access tokens — things you must NEVER put directly in your code files.

& p NEVER paste API keys or passwords into your code files. If your repo is public, anyone can steal them.

6.2 Adding Secrets in Replit

1 Click the padlock icon 🔒

Find it in the left sidebar, labelled "Secrets".

2 Click "+ New Secret"

A form with two fields appears: Key and Value.

3 Enter the Key

This is the name you will use in your code, e.g. MY_API_KEY

4 Enter the Value

Paste your actual secret value here, e.g. sk-abc123...

5 Click "Add Secret"

Done. The secret is encrypted and available to your code as an environment variable.

6.3 Using Secrets in Your Code

JavaScript / Node.js:

```
const apiKey = process.env.MY_API_KEY;  
console.log('Key: ' + apiKey);  
print('Key: ', api_key)
```

Python:

6.4 The .env File

```
MY_API_KEY=sk-abc123yourkeyhere
```

```
DATABASE_URL=postgresql://user:pass@host/db
```

Some projects use a .env file. Create it in your project root and add lines like:

🔒 Add ".env" to a file called ".gitignore" so it is never pushed to GitHub.

```
$ npm install dotenv
```

INSTALL DOTENV (READS YOUR .ENV FILE IN NODE.JS)

```
require('dotenv').config();
```

Include the above code at the top of your main JS file:

Chapter 7: Deploying Your App

Make your app live on the internet for anyone to visit

7.1 What Does "Deploy" Mean?

Right now your app only runs inside Replit — no one else can access it. Deploying publishes your app to a live server with a real public URL that works 24/7, even when you close your browser.

7.2 Deploy with Replit (Easiest Way)

1 Click the "Deploy" button

Find it in the top toolbar of the editor. A deployment panel opens.

2 Choose a deployment type

"Autoscale" is best for web apps — it scales automatically.

"Static" is for HTML/CSS/JS sites with no backend server.

3 Set your run command

This is the command that starts your app, e.g. `npm run start`

4 Click "Deploy"

Replit builds and publishes your app. This takes 1–3 minutes.

5 Get your live URL

Replit gives you a URL like `https://my-app.username.repl.co` — share it with anyone.

Ø=Ü The free plan gives you a repl.co subdomain. Paid plans let you connect a custom domain (e.g. `mysite.com`).

7.3 Saving Your Code to GitHub

GitHub is a website that stores your code and tracks every change you make. This is called "version control". It is the industry standard — every professional developer uses it.

1 Create a free GitHub account

Go to `github.com` and sign up.

2 Create a new repository

Click the green "New" button. Name it the same as your project.

3 Connect in Replit

In your Repl, click the Git icon (branch icon Ø<ß?) in the left sidebar. Sign in to GitHub and link your repository.

4 Commit your work

Type a short message describing what you did (e.g. "Add homepage"), then click "Commit & push".

```
$ git add .
$ git commit -m "Add homepage"
OR DO IT IN THE SHELL — ADD ALL FILES
SAVE A SNAPSHOT WITH A MESSAGE
```

```
$ git push origin main
```

UPLOAD TO GITHUB

📌 Commit at least once per session. If you break something, you can always roll back to a previous save.

Chapter 8: Keyboard Shortcuts & Pro Tips

Work faster without touching the mouse

8.1 Must-Know Shortcuts in Replit

Ctrl + S	Save the current file
Ctrl + Z	Undo last change
Ctrl + Y / Ctrl + Shift + Z	Redo
Ctrl + C / Ctrl + X / Ctrl	Copy / Cut / Paste
Ctrl + A	Select all text in the file
Ctrl + F	Find text in the current file
Ctrl + H	Find and replace text
Ctrl + /	Comment or uncomment selected lines
Ctrl + D	Select next occurrence of highlighted word
Alt + !' / !"	Move current line up or down
Ctrl + Enter	Run the project (same as %⏎ Run button)
Ctrl + Shift + P	Open command palette — search any editor command
Tab	Indent selected code right
Shift + Tab	Un-indent selected code left
Ctrl + G	Go to a specific line number
Ctrl + L	Select the entire current line

8.2 Developer Pro Tips

- Read error messages fully — they tell you the exact file and line number.
- Google the error message in quotes — millions of developers had the same problem.
- Use `console.log()` in JS or `print()` in Python to inspect what a variable contains.
- Save with Ctrl + S before clicking Run — the editor might have unsaved changes.
- If the Preview is blank, check the Console tab for red error messages first.
- Keep functions short — one function should do ONE thing. Over 30 lines? Break it up.
- Commit your code to GitHub at the end of every coding session.
- Comment your code: `//` what this does in JS, or `#` what this does in Python.
- If packages seem broken, restart the Shell: click the "↑ menu ! "Restart shell".
- Test on mobile: resize the preview panel narrow to check your layout.

Chapter 9: Common Errors & How to Fix Them

Every developer hits these — here is how to solve them

9.1 JavaScript / Node.js Errors

```
console.log(myName); // 'L typo
const myName = 'Charlie';
console.log(myName); // ' correct
```

ReferenceError: myVariable is not defined

You used a variable before declaring it, or you misspelled its name. Check the exact spelling.

SyntaxError: Unexpected token

There is a structural typo — a missing bracket, parenthesis, or comma. Look at the line the error points to and check for missing `{}` `()` `[]` or `,`

TypeError: Cannot read properties of undefined (reading 'name')

You tried to use `.name` on a variable that is undefined (empty). Check the variable has data first:

```
Error: Cannot find module 'express'
```

`$ npm install express` package. Go to the Shell and run:

9.2 React Errors

Each child in a list should have a unique "key" prop

When you use `.map()` to render a list, each item needs a key: `<div>{items.map((item) => <li key={item.id}>{item.name}</li>)}</div>`

React Hook "useState" cannot be called conditionally

Hooks must ALWAYS be called at the top of your component function, never inside an if-block or loop.

JSX element has no corresponding closing tag

Every JSX element must be closed. Self-closing tags need a slash: `<br />` `<img />`

9.3 General Quick Fixes

- App not updating after code change !' click %F Run again
- "command not found: node" !' your project is Python-based, not Node.js
- Blank preview, no errors !' check that your start command is correct in the Run config
- "Permission denied" in Shell !' try adding `sudo` before the command
- `node_modules` folder missing !' run `npm install` in the Shell
- Changes not saving !' confirm you are in the right file tab (check the tab name)

Chapter 10: Full Project Build Checklist

Use this every time you start a new application

Phase 1 — Plan (Before you touch code)

Spending 15 minutes planning saves hours of rewriting.

- & Write one sentence: "This app lets users ____"
- & List every page or screen the app needs
- & Sketch a rough layout on paper (boxes are fine)
- & List what data the app stores
- & Choose your stack: plain HTML? React? Node + React? Python?

Phase 2 — Setup

- & Create a new Repl with the right template
- & Connect to a GitHub repo (git init and link in the sidebar)
- & Set up folder structure: src/, public/, components/
- & Install all required packages (npm install ...)
- & Add API keys and passwords to Secrets (padlock icon)

Phase 3 — Build

- & Build the HTML structure first (index.html or App.jsx)
- & Add CSS to style the layout
- & Add JavaScript logic (buttons, forms, data)
- & Connect to any APIs or backend services
- & Test each feature as you finish it — do not wait until the end
- & Commit to GitHub at the end of each coding session

Phase 4 — Test & Fix

- & Click every button and fill out every form in the Preview
- & Open the Console tab and fix every red error
- & Resize the Preview to a phone width and check the layout
- & Fix all errors before deploying

Phase 5 — Deploy

- & Click the "Deploy" button in the top toolbar
- & Set the correct run and build commands
- & Wait for the green "Deployed" status message
- & Visit your live URL and test it end-to-end
- & Share the URL with a friend or family member to get feedback

🎉 You have built and deployed your own app. Every great developer started exactly where you are. Keep building!

You've got this! Ø=Þ€

Every great developer started exactly where you are right now.

Keep building. Keep breaking things. Keep learning.